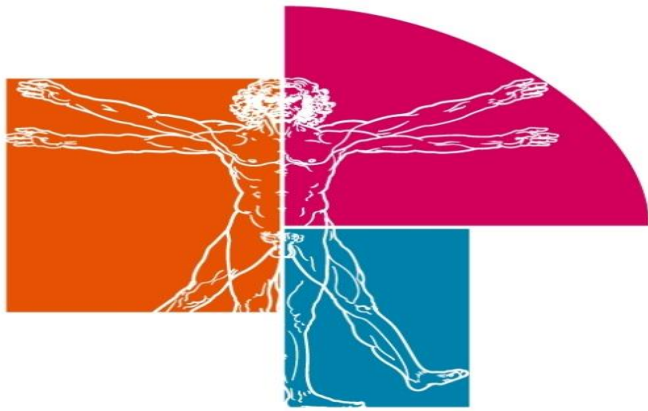




IES LeonardoDaVinci
ALBACETE



TEMA 4 UTILIZACIÓN DE TÉCNICAS DE ACCESO A DATOS CON JAVA



OBJETIVOS

- Analizar las tecnologías que permiten el acceso mediante programación a la información disponible en almacenes de datos.
- Crear aplicaciones que establezcan conexiones con bases de datos relacionales.
- Recuperar información almacenada en bases de datos.
- Publicar en aplicaciones web la información recuperada.
- Crear aplicaciones web que permitan la actualización y la eliminación de información disponible en una base de datos.



INDICE

1. Introducción
2. Conectividad JDBC
3. Establecimiento de conexiones
4. Codificación de aplicaciones con BBDD
5. Tipos SQL en Java



1. Introducción

- **JDBC** (Java DataBase Connectivity) es un API de Java que permite al programador ejecutar instrucciones en lenguaje estándar de acceso a Bases de Datos, SQL (Structured Query Language).
- **SQL** es un lenguaje de muy alto nivel que permite crear, examinar, manipular y gestionar **Bases de Datos relacionales**.
- Para que una aplicación pueda hacer operaciones en una Base de Datos, ha de tener una conexión con ella, que se establece a través de un **driver**, que convierte el lenguaje de alto nivel en sentencias de Base de Datos.

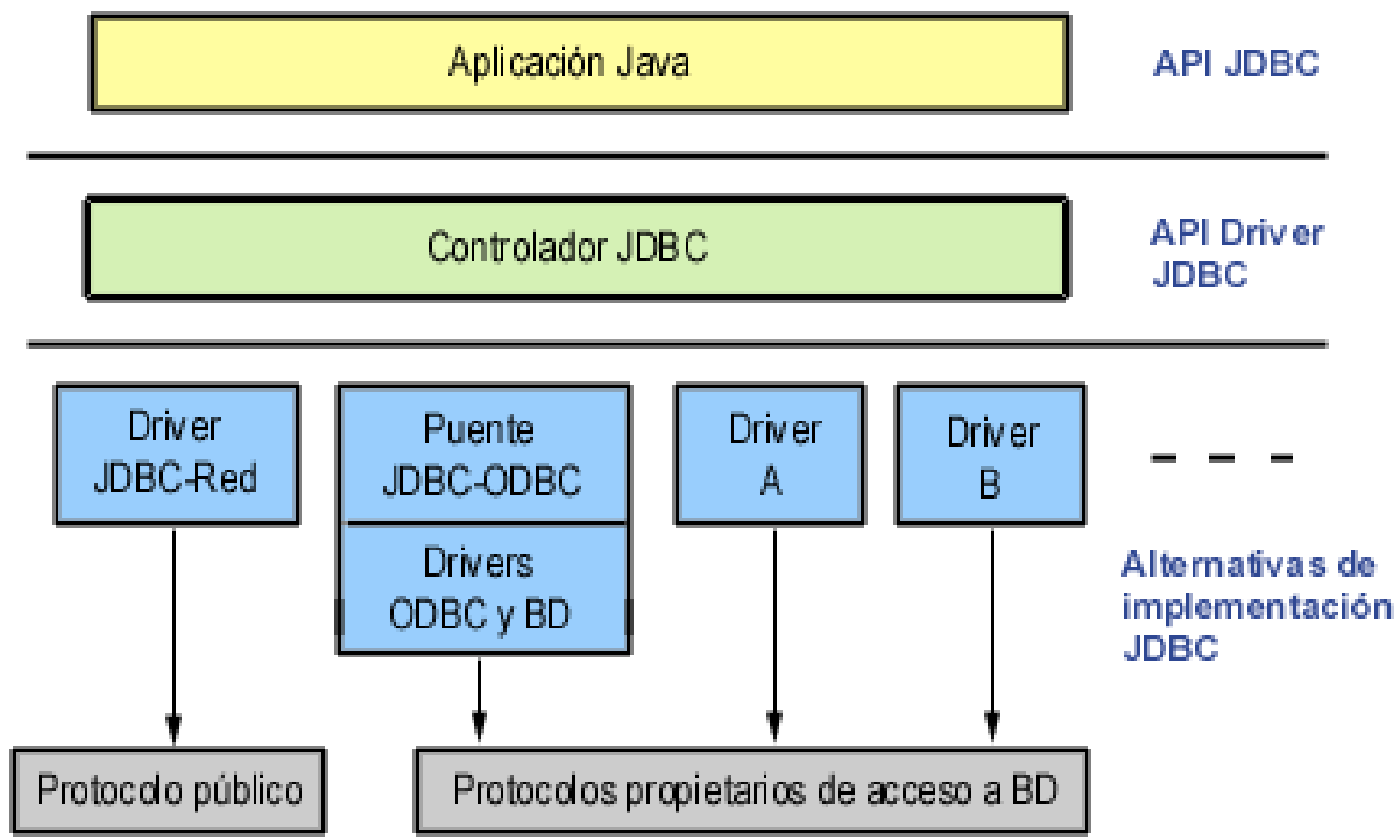


- Las **tres acciones principales** que realizará JDBC son:
 - **Establecer la conexión a una base de datos**, ya sea remota o no.
 - **Enviar sentencias SQL** a esa base de datos.
 - **Procesar los resultados** obtenidos de la base de datos.
- Microsoft creó un driver estandarizado llamado **ODBC** (Open DataBase Connectivity).
- No es un driver rápido ni sofisticado, pero sí asegura el perfecto funcionamiento en plataformas PC-Windows.



2. Conectividad JDBC

- JDBC es una especificación de un **conjunto de clases y métodos de operación** que **permiten a cualquier programa Java acceder a sistemas de bases de datos** de forma homogénea.
- Para ello **la aplicación Java debe tener acceso a un driver JDBC adecuado.**
- Este driver es el que **implementa la funcionalidad de todas las clases de acceso a datos y proporciona la comunicación entre el API JDBC y la base de datos real.**





- **El API JDBC viene distribuido en dos paquetes:**
 - **java.sql**, dentro de J2SE
 - **javax.sql**, extensión dentro de J2EE
- Cuando se construye una aplicación de base de datos, JDBC oculta los detalles específicos de cada base de datos, de modo que el programador se ocupe solo de su aplicación.
- **El conector lo proporciona el fabricante de la base de datos o bien un tercero.**



- JDBC ofrece las clases e interfaces para:
 - **Establecer una conexión a una base de datos.**
 - **Ejecutar una consulta.**
 - **Procesar los resultados.**
- Ejemplo: Código Java para establecer una conexión y ejecutar consulta:



// Establece la conexión

```
Connection con = DriverManager.getConnection ("jdbc:odbc:miBD", "miLogin", "miPassword");
```

// Ejecuta la consulta

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery("SELECT nombre, edad FROM Jugadores");
```

// Procesa los resultados

```
while (rs.next()) {  
    String nombre = rs.getString("nombre");  
    int edad = rs.getInt("edad");  
}
```



3. Establecimiento de conexiones

- JDBC define ocho interfaces para operaciones con bases de datos, de las que se derivan las clases correspondientes.
- La clase que se encarga de cargar inicialmente todos los drivers JDBC disponibles es **DriverManager**.
- Una aplicación puede utilizar DriverManager para obtener un objeto de tipo conexión, **Connection**, con una base de datos. La conexión se especifica siguiendo la sintaxis:

`jdbc:subprotocolo//servidor:puerto/base de datos`



- Una vez que se tiene un objeto de tipo **Connection**, se pueden crear sentencias, **Statements**, ejecutables.
- Cada una de estas sentencias puede devolver cero o más resultados, que se devuelven como objetos de tipo **ResultSet**.
- La tabla siguiente muestra la lista de clases e interfaces junto con una breve descripción.



<i>Clase/Interface</i>	<i>Descripción</i>
Driver	Permite conectarse a una base de datos: cada gestor de base de datos requiere un driver distinto
DriverManager	Permite gestionar todos los drivers instalados en el sistema
DriverPropertyInfo	Proporciona diversa información acerca de un driver
Connection	Representa una conexión con una base de datos. Una aplicación puede tener más de una conexión a más de una base de datos
DatabaseMetadata	Proporciona información acerca de una Base de Datos, como las tablas que contiene, etc.



Statement	Permite ejecutar sentencias SQL sin parámetros
PreparedStatement	Permite ejecutar sentencias SQL con parámetros de entrada
CallableStatement	Permite ejecutar sentencias SQL con parámetros de entrada y salida, típicamente procedimientos almacenados
ResultSet	Contiene las filas o registros obtenidos al ejecutar un SELECT
ResultSetMetadata	Permite obtener información sobre un ResultSet , como el número de columnas, sus nombres, etc.



- Si no se encuentra ningún driver adecuado, se lanza una SQLException.
- Código Java para establecer una conexión con MySQL. :

```
public static void main(String[] args) {  
    try {  
        // Cargar el driver de mysql, Java puede entenderse con MySQL.  
        Class.forName("com.mysql.jdbc.Driver");  
        /* Cadena de conexión para conectar con MySQL en localhost,  
        seleccionar la base de datos llamada "ejercicio"*/  
        String connectionUrl = "jdbc:mysql://localhost/ejercicio";  
        // Obtener la conexión con usuario y contraseña del servidor de MySQL: root y root  
        Connection con = DriverManager.getConnection(connectionUrl,"root","root");  
    } catch (SQLException e) {  
        System.out.println("SQL Exception: "+ e.toString());  
    } catch (ClassNotFoundException cE) {  
        System.out.println("Excepción: "+ cE.toString());  
    }  
}
```



4. Codificación de aplicaciones con BBDD

○ Los pasos a seguir son:

1. Descargar el controlador.

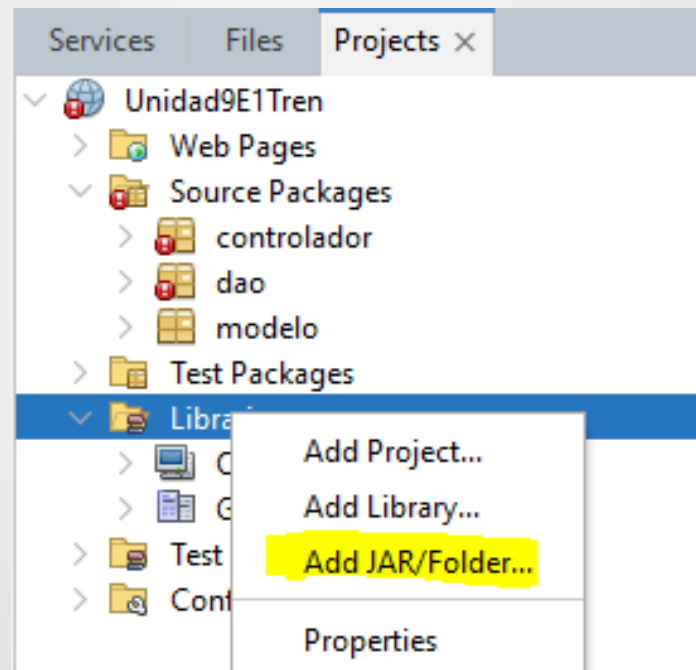
- ✓ Descargarnos el conector o driver que necesitamos para trabajar con MySQL desde su página oficial.

<https://dev.mysql.com/downloads/connector/j/>

Una vez descargado hay que descomprimirlo y buscar el archivo .jar que será el que añadiremos desde NetBeans a las librerías de nuestro proyecto.

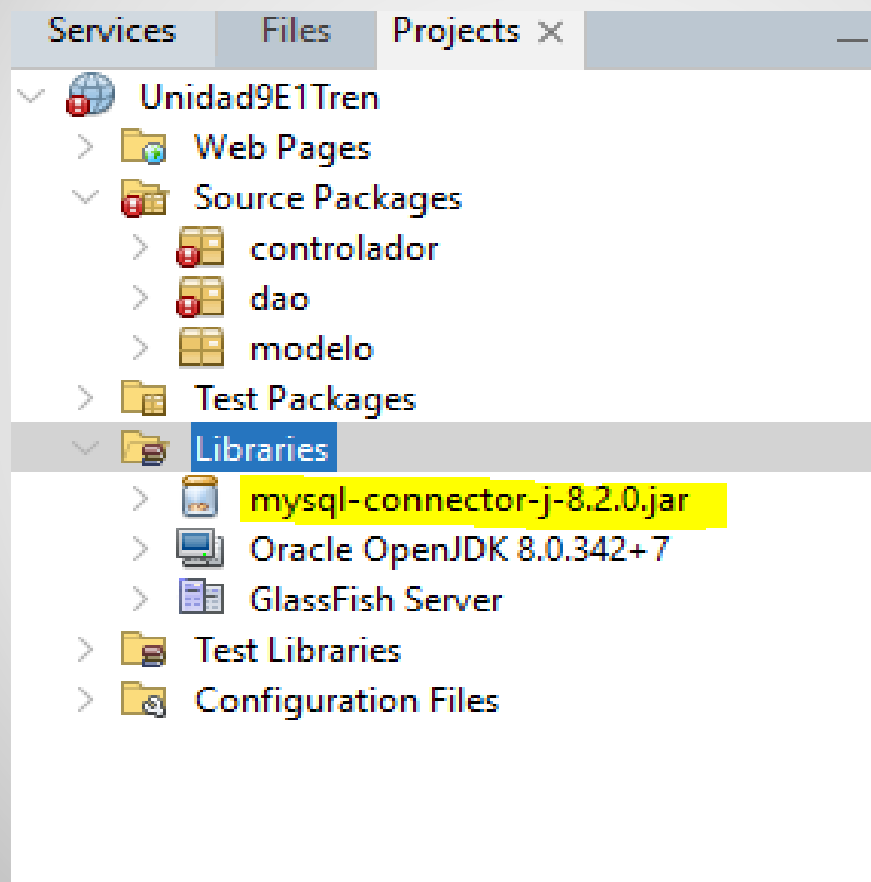


- ✓ Sobre nuestro proyecto -> Libraries -> click derecho -> Add JAR/Folder





✓ Buscarlo donde lo tengamos guardado y añadirlo





- ✓ Otra opción menos extendida pero más eficiente es copiar el archivo .jar en las librerías externas de la carpeta .jre del jdk que estemos utilizando.
- ✓ En mi caso C:\Program Files\Java\jdk Java Web Oracle\openjdk-8u342-b07\jre\lib\ext

Este equipo > Windows8_OS (C:) > Archivos de programa > Java > jdk Java Web Oracle > openjdk-8u342-b07 > jre > lib > ext				
Nombre	Fecha de modificación	Tipo	Tamaño	
access-bridge-64	17/07/2022 0:14	Executable Jar File	193 KB	
cldrdata	17/07/2022 0:14	Executable Jar File	3.771 KB	
dnsns	17/07/2022 0:14	Executable Jar File	9 KB	
jaccess	17/07/2022 0:14	Executable Jar File	44 KB	
localedata	17/07/2022 0:14	Executable Jar File	1.156 KB	
meta-index	17/07/2022 0:15	Archivo	1 KB	
mysql-connector-j-8.2.0	18/09/2023 18:12	Executable Jar File	2.432 KB	
nashorn	17/07/2022 0:14	Executable Jar File	1.987 KB	
sunec	17/07/2022 0:14	Executable Jar File	38 KB	
sunjce_provider	17/07/2022 0:14	Executable Jar File	270 KB	



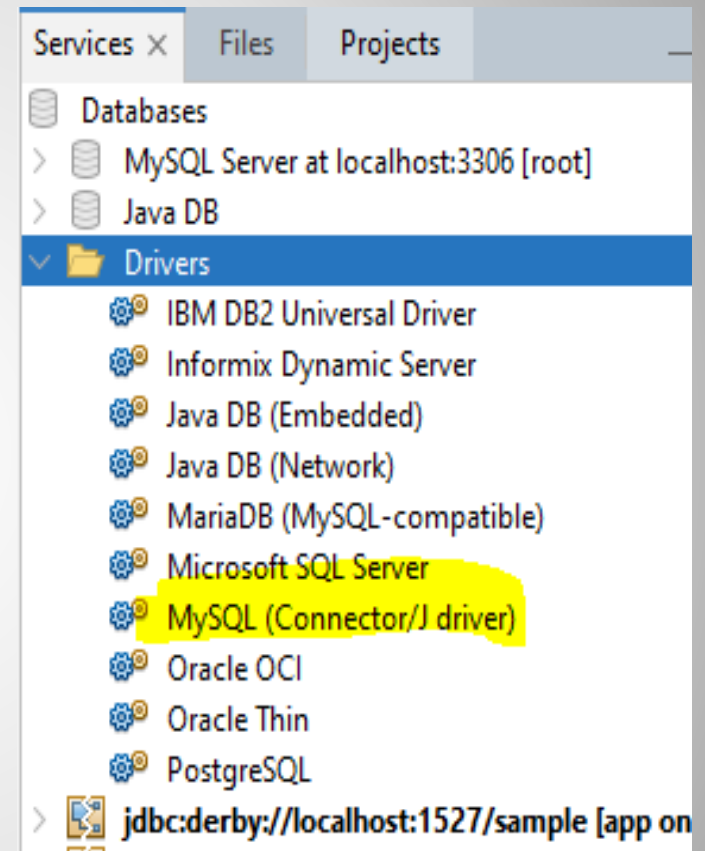
2. Registrar el controlador JDBC

- ✓ Registrar el controlador que queremos utilizar es tan fácil como escribir una línea de código.
- ✓ Hay que consultar la documentación del controlador que vamos a utilizar para conocer el nombre de la clase que hay que emplear. En el caso del controlador para MySQL es **"com.mysql.cj.jdbc.Driver"**, o sea, que se trata de la clase Driver que está en el paquete com.mysql.cj.jdbc del conector que hemos descargado, y que no es más que una librería empaquetada en un fichero .jar.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

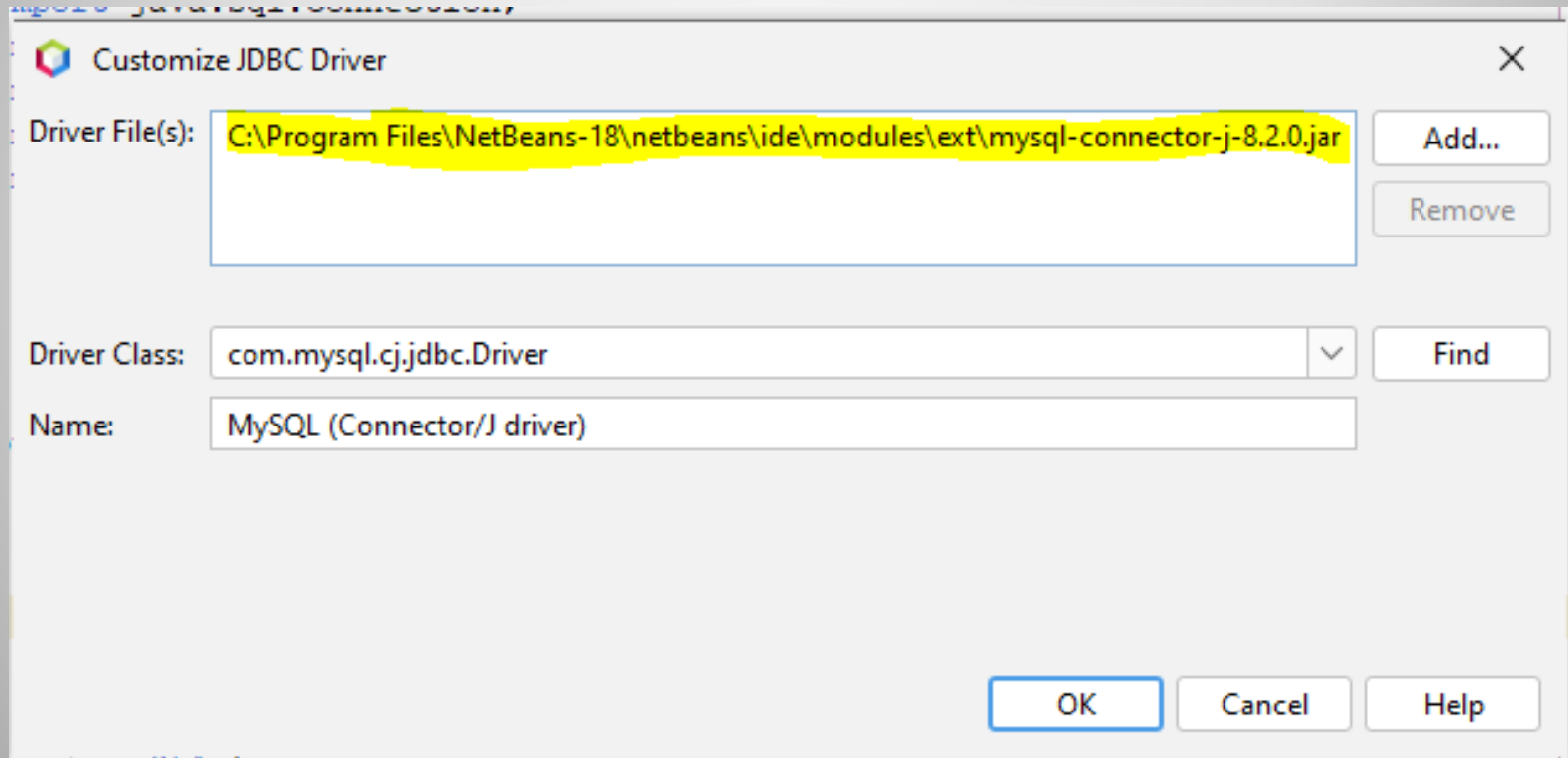



- ✓ En el caso de que no se encuentre la ruta al Driver o que la hayamos puesto mal, nos debemos asegurar de ponerlo en una ruta accesible desde Netbeans. Esto lo podemos ver desde el menú Windows -> Services -> Databases -> Drivers -> posicionarse sobre MySQL (Connector/J driver)





- ✓ Botón derecho -> Customize y comprobar que la ruta es accesible y si no, crear una nueva y eliminar la otra





3. Ejecución de consultas sobre la BBDD

- ✓ Para operar con una base de datos ejecutando las consultas necesarias, nuestra aplicación deberá hacer las operaciones siguientes:
 - Cargar el conector necesario para comprender el protocolo que usa la base de datos en cuestión.
 - Establecer una conexión con la base de datos.
 - Enviar consultas SQL y procesar el resultado.
 - Liberar los recursos al terminar.
 - Gestionar los errores que se puedan producir.



- ✓ Podemos utilizar los siguientes tipos de sentencias:
- **Statement:** para sentencias sencillas en SQL.
- **PreparedStatement:** para consultas preparadas, como por ejemplo las que tienen parámetros.
- **CallableStatement:** para ejecutar procedimientos almacenados en la base de datos.



- ✓ El API JDBC distingue dos tipos de consultas:
- **Consultas:** SELECT. Para las sentencias de consulta que obtienen datos de la base de datos, se emplea el método **ResultSet executeQuery(String sql)**. El método de ejecución del comando SQL devuelve un objeto de tipo ResultSet que sirve para contener el resultado del comando SELECT, y que nos permitirá su procesamiento.
- **Actualizaciones:** INSERT, UPDATE, DELETE, sentencias DDL. Para estas sentencias se utiliza el método **int executeUpdate(String sql)**.



4. Recuperación de información

- ✓ Las consultas a la base de datos se realizan con sentencias SQL que van "embebidas" en otras sentencias especiales que son propias de Java.
- ✓ Las consultas **devuelven un ResultSet**, que es una clase java parecida a una lista en la que se aloja el resultado de la consulta. **Cada elemento de la lista es uno de los registros de la base de datos** que cumple con los requisitos de la consulta.
- ✓ El objeto ResultSet contiene la tabla resultado de la pregunta SQL que se haya realizado.



resultado →

nif	nombre	Apellidos	Teléfono
11.111.111	Carlos	Escobar Pérez	654897654
22.222.222	Elena	Pulido López	645321200
33.333.333	Gloria	Fiz García	664231209
44.444.444	Alicia	Sánchez Font	675443312

- ✓ Con el **ResultSet** hay disponibles una serie de **métodos** que **permiten movernos hacia delante y hacia atrás en las filas**, y **obtener la información de cada fila**.
- ✓ Cuando se obtiene la tabla resultado, no podemos leer directamente de la misma, **necesitamos un next() para situarnos en el primer elemento válido**.



- ✓ Por ejemplo, para obtener: nif, nombre, apellidos y teléfono de los clientes que están almacenados en la tabla del mismo nombre, haríamos la siguiente consulta:

// Preparamos la consulta y la ejecutamos

```
Statement s = n.createStatement();
```

```
ResultSet rs = s.executeQuery ("SELECT NIF,  
NOMBRE,APELLIDOS, TELÉFONO FROM CLIENTE");
```

- ✓ El **método next() del ResultSet** hace que dicho puntero avance al siguiente registro. Si lo consigue, el método next() devuelve true. Si no lo consigue, porque no haya más registros que leer, entonces devuelve false.



- ✓ El método `executeQuery` devuelve un objeto `ResultSet` para poder recorrer el resultado de la consulta utilizando un cursor.
- ✓ Para **obtener una columna del registro** utilizamos los **métodos `get`**. Hay un método `get...` para cada tipo básico Java y para las cadenas.
- ✓ Cuando trabajamos con el `ResultSet`, en cada registro, los métodos `getInt()`, `getString()`, `getDate()`, etc., nos devuelve los valores de los campos de dicho registro. Podemos pasar a estos métodos un índice (que comienza en 1) para indicar qué columna de la tabla de base de datos deseamos, o bien, podemos usar un `String` con el nombre de la columna (tal cual está en la tabla de base de datos).



✓ Ejemplo:

Código consulta e iteración.

// Obtener la conexión

```
Connection con = DriverManager.getConnection(connectionUrl);
```

// Preparamos la consulta

```
Statement s = con.createStatement();
```

```
ResultSet rs = s.executeQuery
```

```
("SELECT NIF, NOMBRE, APELLIDOS, TELÉFONO FROM CLIENTE");
```

// Iteramos sobre los registros del resultado

```
while (rs.next())
```

```
System.out.println (rs.getString("NIF") + " " +
```

```
rs.getString (2) + " " +
```

```
rs.getString (3) + " " +
```

```
rs.getString (4));
```



5. Actualización de información

- ✓ Respecto a las consultas de actualización, **executeUpdate**, retornan el número de registros insertados, registros actualizados o eliminados, dependiendo del tipo de consulta que se trate.
- ✓ Supongamos que tenemos varios registros en una tabla Cliente, de una base de datos. Si quisiéramos actualizar el teléfono del tercer registro, que tiene idCLIENTE=3 y ponerle como nuevo teléfono el 968610009 tendríamos que hacer:



```
String connectionUrl = "jdbc:mysql://localhost/notarbd" ;  
// Obtener la conexión  
Connection con = DriverManager.getConnection(connectionUrl,"root","root");  
// Preparamos la consulta y la ejecutamos  
Statement s = con.createStatement();  
s.executeUpdate("UPDATE CLIENTE SET teléfono='968610009' WHERE idCLIENTE=3");  
// Cerramos la conexión a la base de datos.  
con.close();
```




6. Añadir información

- ✓ Si queremos añadir un registro a la tabla Cliente, de la base de datos con la que estamos trabajando tendremos que utilizar la sentencia INSERT INTO de SQL. Al igual que hemos visto en el apartado anterior, utilizaremos executeUpdate pasándole como parámetro la consulta, de inserción en este caso.
- ✓ Para insertar el contenido de una variable de tipo VARCHAR se debe introducir entre comillas simples ‘ ‘.



// Preparamos la consulta y la ejecutamos

```
Statement s = con.createStatement();
```

```
s.executeUpdate( "INSERT INTO CLIENTE" +
```

```
" (idCLIENTE, NIF, NOMBRE, APELLIDOS, DIRECCIÓN, CPOSTAL, TELÉFONO,
```

```
CORREOELEC)" + " VALUES (4, '66778998T', 'Alfredo', 'Gates Gates', 'C/ Pirata 23', '20400',
```

```
'891222112', 'prueba@eresmas.es' )" );
```



7. Borrado de información

- ✓ Cuando nos interese eliminar registros de una tabla de una base de datos, emplearemos la sentencia SQL: DELETE.
- ✓ Así, por ejemplo, si queremos eliminar el registro a la tabla Cliente, de nuestra base de datos correspondiente a la persona que tiene el nif: 66778998T, tendremos que utilizar el código:

```
// Preparamos la consulta y la ejecutamos
Statement s = con.createStatement();
numReg = res.executeUpdate( "DELETE FROM CLIENTE WHERE NIF= '66778998T' " );
// Informamos del número de registros borrados
System.out.println ( "\nSe borró " + numReg + " registro\n" );
```



8. Cierre de conexiones

- ✓ Las conexiones a una base de datos consumen muchos recursos en el sistema gestor y por tanto en el sistema informático en general. Por ello, conviene cerrarlas con el método **close()** siempre que vayan a dejar de ser utilizadas, en lugar de esperar a que el 'garbage collector' de Java las elimine.
- ✓ También conviene cerrar las consultas (Statement y PreparedStatement) y los resultados (ResultSet) para liberar los recursos.



9. Excepciones en JDBC

- ✓ En todas las aplicaciones en general, y por tanto en las que acceden a bases de datos en particular es conveniente emplear el método **getMessage()** y **getErrorCode()** de la clase **SQLException** para recoger y mostrar el mensaje de error que ha generado MySQL, lo que seguramente nos proporcionará una información más ajustada sobre lo que está fallando.
- ✓ Cuando se produce un error se lanza una excepción del tipo **java.sql.SQLException**.
- ✓ Es importante que las operaciones de acceso a base de datos estén dentro de un bloque **try catch** que gestione las excepciones.



10. Sentencias preparadas para evitar SQLInjection

- ✓ Es conveniente utilizar sentencias preparadas sobre todo cuando se van a actualizar datos.
- ✓ Las sentencias preparadas son aquellas en donde los datos van sustituidos por ? , y posteriormente se describen con los distintos set de cada uno de los tipos de datos utilizados.
- ✓ La sentencia preparada debe ser de tipo PreparedStatement asociado a la cadena que describe la sentencia a ejecutar.



```
public void insertarVotante(Votante _ObjVotante, ..... ) {  
    try{  
        //Sentencias preparadas  
        String ordenSQL="INSERT INTO VOTANTE VALUES(null,?,?,?,?,?,?'N','Votante)";  
        PreparedStatement PrepStm=_Conexion.prepareStatement(ordenSQL);  
        PrepStm.setString(1,_ObjVotante.getNif());  
        PrepStm.setString(2,_ObjVotante.getNombre());  
        PrepStm.setString(3,_ObjVotante.getApellidos());  
        PrepStm.setString(4,_ObjVotante.getDomicilio());  
        PrepStm.setDate(5,java.sql.Date.valueOf(_ObjVotante.getFechaNac()));  
        PrepStm.setString(6,_ObjVotante.getPassword());  
        int filas=PrepStm.executeUpdate();  
        //Comprobar si se ha insertado  
        if(filas!=1){  
            ..... }  
        } catch(SQLException SQLE){  
            String MensajeError=SQLE.getMessage();  
            intCodigoError=SQLE.getErrorCode();  
        }  
  
    }//insertarVotante
```



5. Tipos SQL en Java

Java	SQL
String	VARCHAR
boolean	BIT
byte	TINYINT
short	SMALLINT
int	INTEGER
long	BIGINT
float	REAL
double	DOUBLE
byte[]-byte array: imágenes, sonidos...	VARBINARY (BLOBs)
java.sql.Date	DATE
java.sql.Time	TIME
java.sql.Timestamp	TIMESTAMP
java.math.BigDecimal	NUMERIC



La conversión al contrario sería:

SQL	Java
CHAR	String
VARCHAR	String
LONGVARCHAR	String
NUMERIC	java.math.BigDecimal
DECIMAL	java.math.BigDecimal
BIT	boolean
TINYINT	byte
SMALLINT	short
INTEGER	int
BIGINT	long
REAL	float



SQL	Java
FLOAT	double
DOUBLE	double
BINARY	byte[]
VARBINARY	byte[]
LONGVARBINARY	byte[]
DATE	java.sql.Date
TIME	java.sql.Time
TIMESTAMP	java.sql.Timestamp